

1 Проверка за столом

1.1 Описание тестируемой программы

Название: построение двумерного клеточного автомата с окрестностью фон Неймана

Дано:

- `width` - ширина поля
- `height` - высота поля
- `choice` - режим заполнения начального состояния
- `iterations` - число итераций

Требуется: моделирование эволюции клеточного автомата по заданному количеству итераций по заданной функции, где состояние каждой клетки зависит от её соседей (слева, сверху, справа, снизу) и текущего значения.

Ограничения: все параметры - целые числа:

- $width \in [1, 80]$
- $height \in [1, 80]$
- $choice \in (m, r)$
- $iterations \in [1, \infty)$

1.2 Спецификация

Входные данные	Выходные данные	Реакция программы
width = 5, height = 5, choice = r, iterations = 6	Заполненный массив, выведенный на экран с помощью нулей и единиц.	Случайным образом за- полняется поле заданного размера. По заданной функции в программе строится дальнейшее поведение клеточного ав- томата исходя из соседних элементов для каждого элемента поля. На каждом шаге t состояние каждой клетки обновляется со- гласно правилам перехода в зависимости от заданной функции.
width = 5, height = 5, choice = m, iterations = 6	Заполненный массив, выведенный на экран с помощью нулей и единиц.	Пользователем заполняет- ся поле заданного разме- ра. По заданной функции в программе строится даль- нейшее поведение клеточ- ного автомата исходя из со- седних элементов для каж- дого элемента поля. На каждом шаге t состояние каждой клетки обновляет- ся согласно правилам пере- хода в зависимости от за- данной функции.
width = 85	Сообщение на экране: «Ошибка: ширина должна быть целым числом от 1 до 80.»	Повторный запрос ввода от пользователя до тех пор, пока не будет введено корректное значение.
width = 3, height = 85	Сообщение на экране: «Ошибка: высота должна быть целым числом от 1 до 80.»	Повторный запрос ввода от пользователя до тех пор, пока не будет введено корректное значение.

1.3 Исходный код

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <random>
5 #include <iomanip>
6 #include <algorithm>
7
8 using namespace std;
9
10 const string RULE_STRING = "000010010001011000011110100000000";
11
12 int getNextState(int s0, int s1, int s2, int s3, int s4) {
13     int index = (s0 << 4) | (s1 << 3) | (s2 << 2) | (s3 << 1) | s4;
14     return RULE_STRING[index] - '0';
15 }
16
17 class CellularAutomaton {
18     private:
19         int width, height;
20         vector<vector<int>> currentGrid;
21         vector<vector<int>> nextGrid;
22
23         int toroidal(int x, int size) {
24             return ((x % size) + size) % size;
25         }
26
27         int getNeighbor(int row, int col, int dr, int dc) {
28             int nr = toroidal(row + dr, height);
29             int nc = toroidal(col + dc, width);
30             return currentGrid[nr][nc];
31         }
32
33     public:
34         CellularAutomaton(int w, int h) : width(w), height(h) {
35             currentGrid.resize(height, vector<int>(width, 0));
36             nextGrid.resize(height, vector<int>(width, 0));
37         }
38
39         void setManualInitialState() {
40             cout << "Введите состояние каждой клетки в строке через пробел (0 или 1):\n";
41             for (int i = 0; i < height; ++i) {
42                 cout << "Строка " << i + 1 << ": ";
43                 for (int j = 0; j < width; ++j) {
44                     cin >> currentGrid[i][j];
45                     if (currentGrid[i][j] != 0 && currentGrid[i][j] != 1) {
46                         cout << "Ошибка: введите только 0 или 1.\n";
47                         j--;
48                     }
49                 }
50             }
51         }
52     };
53 }
```

```

50     }
51 }
52
53 void setRandomInitialState() {
54     random_device rd;
55     mt19937 gen(rd());
56     uniform_int_distribution<> dis(0, 1);
57
58     for (int i = 0; i < height; ++i) {
59         for (int j = 0; j < width; ++j) {
60             currentGrid[i][j] = dis(gen);
61         }
62     }
63 }
64
65 void update() {
66     for (int i = 0; i < height; ++i) {
67         for (int j = 0; j < width; ++j) {
68             int s0 = currentGrid[i][j];
69             int s1 = getNeighbor(i, j, -1, 0); // верх
70             int s2 = getNeighbor(i, j, 0, 1); // право
71             int s3 = getNeighbor(i, j, 1, 0); // низ
72             int s4 = getNeighbor(i, j, 0, -1); // лево
73
74             nextGrid[i][j] = getNextState(s0, s1, s2, s3, s4);
75         }
76     }
77     currentGrid = nextGrid;
78 }
79
80 void print() const {
81     for (int i = 0; i < height; ++i) {
82         for (int j = 0; j < width; ++j) {
83             cout << currentGrid[i][j] << " ";
84         }
85         cout << endl;
86     }
87     cout << "-----" << endl;
88 }
89
90 void run(int iterations) {
91     cout << "\nНачальное состояние:\n";
92     print();
93
94     for (int iter = 1; iter <= iterations; ++iter) {
95         cout << "Итерация " << iter << ":\n";
96         update();
97         print();
98     }
99 }
100 };
101

```

```

102 int main() {
103     int width, height, iterations;
104     char choice;
105     do {
106         cout << "Введите ширину поля (от 1 до 80): ";
107         cin >> width;
108         if (cin.fail() || width < 1 || width > 80) {
109             cout << "Ошибка: ширина должна быть целым числом от 1 до
110                 80.\n";
111             cin.clear();
112             cin.ignore(10000, '\n');
113             width = -1;
114         }
115     } while (width < 1 || width > 80);
116     do {
117         cout << "Введите высоту поля (от 1 до 80): ";
118         cin >> height;
119         if (cin.fail() || height < 1 || height > 80) {
120             cout << "Ошибка: высота должна быть целым числом от 1 до
121                 80.\n";
122             cin.clear();
123             cin.ignore(10000, '\n');
124             height = -1;
125         }
126     } while (height < 1 || height > 80);
127     CellularAutomaton ca(width, height);
128     do {
129         cout << "Выберите начальные условия:\n";
130         cout << "m - вручную\n";
131         cout << "r - случайно\n";
132         cin >> choice;
133         if (cin.fail()) {
134             cin.clear();
135             cin.ignore(10000, '\n');
136             choice = '\0';
137         }
138     } while (choice != 'm' && choice != 'M' && choice != 'r' &&
139             choice != 'R');
140     if (choice == 'm' || choice == 'M') {
141         ca.setManualInitialState();
142     } else {
143         ca.setRandomInitialState();
144     }
145     do {
146         cout << "Введите количество итераций (от 1 и больше): ";
147         cin >> iterations;
148         if (cin.fail() || iterations < 1) {
149             cout << "Ошибка: количество итераций должно быть целым числ
150                 ом >= 1.\n";
151             cin.clear();

```

```
150         cin.ignore(10000, '\n');
151         iterations = -1;
152     }
153 } while (iterations < 1);
154 ca.run(iterations);
155 return 0;
156 }
```